

Mutation Testing of Simulink Cyber-Physical System Models: Challenges and Solutions in Practice*

Murat Kavak
Universiteit Antwerpen
Belgium
Flanders Make VZW
Belgium

Onur Kilincceker
Universiteit Antwerpen
Belgium
Flanders Make VZW
Belgium

Serge Demeyer
Universiteit Antwerpen
Belgium
Flanders Make VZW
Belgium

Kevin Vandembroucke
DUCO (NV)
Belgium

Halim Abdurrahman Ceylan
Department of International
Computer, Graduate School of
Natural and Applied Sciences, Ege
University, Izmir
Türkiye

Abstract

Several experience reports illustrate that mutation testing is capable of supporting a “shift-left” testing strategy, especially in industries where late bug discovery incurs very high costs and risks. In the context of cyber-physical systems, a “shift left” implies that engineers need to test the design models used to simulate, prototype, and analyse the feasibility of the system under design. In this paper, we analyse the challenges we encountered and the lessons we learned when incorporating mutation testing in the context of DUCO, a company producing ventilation systems for buildings. The engineers within DUCO have years of experience with model-based engineering centered around Simulink and StateFlow, including unit tests for their models. During a pilot project with our tool prototype MUT4SLX we learned that equivalent mutants, requirement traceability, and mutation testing for Stateflow represent particular challenges not yet reported in the academic literature.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**.

Keywords

Mutation Testing, Simulink, Stateflow, MUT4SLX, Industry Experience, Mutation Testing in Practice

1 Introduction

Safety-critical industries (such as automotive and aerospace) mainly rely on model-driven engineering to tackle the complexity and pervasiveness of safety-critical software. This reliance is even more apparent in cyber-physical systems, where software interacts with physical components and processes under strict safety, timing, and reliability constraints [17].

Testing cyber-physical systems is very challenging due to their complexity and heterogeneity, characterized by dependencies among software, physical components, and processes. Shift-left testing is

a software development approach that aims to move testing activities into an early phase of the lifecycle, when dependencies and interactions are simulated using executable models of the system and its environment [14]. This approach is coined as model-based shift-left testing [20], which offers several advantages for ensuring the safety and reliability of modern cyber-physical systems. First, the system is modeled at a higher level of abstraction to manage complexity. Second, since it is an early phase, the cost of fixing is 10 to 30 times lower than that of a bug detected in later development stages [25].

Today, MathWorks Simulink® and Stateflow® offer an advanced and widely used platform for simulation and development of cyber-physical systems [18]. Also, it enables the application of model-based shift-left testing principles to cyber-physical systems. Simulink models allow developers to design systems using block diagrams that transform input signals into output signals, thereby capturing the behavior of complex and dynamic systems. Stateflow, on the other hand, is more effectively used to model and simulate decision logic using state machines and flowcharts. The best practice is to use Stateflow for control and decision logic, alongside Simulink, to develop a cyber-physical system. This way, Simulink’s capabilities are extended to event-driven and hybrid forms of embedded control via Stateflow.

Mutation testing is a recommended practice in industrial standards in automotive, industrial robots, and medical devices for functional safety, such as ISO 26262. The main purpose of mutation testing is to diagnose and address weaknesses in software test suites, typically expressed in textual programming languages such as C++ [12]. This method intentionally injects faults (mutants) into the code or model and measures how many are detected by the test suite, producing a mutant score that reflects the suite’s quality [3]. Based on shift-left testing principles, mutation testing at the early design model-level stage offers advantages over code-level testing, since the system is simpler and the detected faults are less costly and easier to fix. In the literature, it is commonly referred to as model-based mutation testing [1, 6]. To automate this approach for Simulink and Stateflow, SIMULTATE [24], FIM [4], MUT4SLX [9, 22], and BERTiMuS [31] are proof-of-concept tools, each focusing on different aspects and using different methods.

*Accepted for publication at the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE 2026), Munich, Germany. Version of Record: <https://doi.org/10.1145/3832783.3834529>

While these proof-of-concept tool demonstrate the technical feasibility of model-based mutation testing, actual experience reports on how the corresponding tools would work in practice are still lacking. In this paper, we analyse the challenges we encountered and the lessons we learned when incorporating mutation testing in the context of DUCO. DUCO adopted model-based engineering practices as early as 2019. At the time of writing, an average project incorporates 20% Simulink® and 80% Stateflow® models. Unit testing is heavily promoted within the DUCO; by means of the Signal Builder (now Signal Editor)¹ block for test creation in combination with an Assertion block to check whether the actual result matches. DUCO measures the coverage of their unit tests but has noticed that defects still appear later in the life cycle. In this context, they were eager to validate our tool prototype MUT4SLX within their workflow.

The remainder of this paper is organised as follows. Section 2 gives an overview of the state of the art in mutation testing for Simulink and Stateflow models. Section 3.3 describes the research questions, which naturally leads to Section 4 reporting the results. Section 5 presents lessons learned, including practical insights, and concludes the paper in Section 6.

2 Related Work

Fault injection is used to test the robustness of the system under test, whereas mutation testing is used to measure test effectiveness. Both methods inject faults in a systematic way, based on well-defined and structured fault patterns. The fault patterns are (ideally) defined based on specific bug databases to represent realistic errors unintentionally injected by developers (the Competent Programmer hypothesis [8]), and these errors may couple and lead to complex failures (the Coupling Effect hypothesis [8]). Yadav et al. presented a comprehensive analysis and comparison of state-of-the-art fault-injection tools for Simulink Cyber Physical System models, including insights and future direction of research in this research area [29].

Mutation testing for Simulink models dates back to 2005, when Zhan and Clark introduced three types of mutation operators (add, multiply, and assign) for the signal carried on the input by/with a certain value [30]. Those operators are primarily used for search-based test generation in combination with mutation testing. Brillout et al. employed mutation testing again to generate bounded model-checking-based test data for Simulink models [7]. Complementary to the above, there are a few experience reports concerning mutation testing (and the companion technique fault injection) in model-based engineering. Rana et al. presented the results of the first ISO 26262-compliant verification and validation efforts, combining fault injection and mutation testing for Simulink models [26]. This was a collaborative research between the University of Gothenburg and Volvo Car Corporation in the automotive industry. They concluded that fault injection and model-level mutation testing (as part of shift-left testing) add value to the development of high-quality, reliable, and dependable software in the automobile industry, help keep product costs lower, and, most importantly, ensure that automobiles are safer than ever before.

Early research on developing a taxonomy of classes for Simulink model mutations proposed 4 categories to analyze simple faults that designers may commit [15]. Then, a more extensive set of 8 proposed categories was devised to test Simulink model-clone detectors [27]. Table 1 shows those classes with underlying change logic, including corresponding related work fitting to those classes.

Table 1: Simulink Mutation Classes [27]

Mutation Key	Title	Related Work
mMLA	Modification of Layout Attribute	
mRUE	Reordering Underlying Elements	[19, 24]
mRBL	Renaming a Block or Line	
mCBV	Changing a Block's Value	[2, 4, 9, 16, 19, 21, 22, 24]
mADBD	Add or Delete Block as Destination	[19]
mADBS	Add or Delete Block as Source	
mCBT	Changing a Block's Type	[4]
mCSCH	Changing a Subsystem's clone hierarchy	

The first prototype tool, SIMULTATE, automates mutation testing for Simulink models [24]. It relies on a Python interface to automate processes, allowing users to expand with additional mutation operators (initially proposing 4) and to support fault injection (including 5 fault models). Its operators fall into the category of mRUE and mCBV (shown in Table 1). Then, Hanh et al. presented a test data generation approach for Simulink models based on mutation analysis, proposing 10 mutation operators that fall into the mCBV category [16]. Those operators are also implemented in a tool using feature diagrams (inspired by Devroey et al. [13]) to generate mutants for configurable Simulink models [2]. Nanda et al. developed the IMAT tool, which integrates with Simulink Design Verifier (automatic test generator) to connect mutation testing to the model checker [21]. Matinnejad et al. presented a test generation algorithm and a test prioritization algorithm for Simulink models, using mutation testing to measure the fault-revealing abilities of the proposed algorithms [19]. A comprehensive list of Simulink and Stateflow fault patterns was also presented and compared with both the existing literature on mutation operators and the identified ones by Delphi Automotive.

FIM is a proof-of-concept tool that inherits the mutation operators of the SIMULATE tool and supports ten fault injection operators for signals and five mutation operators for blocks [4]. The tool primarily focuses on safety analysis through extensive fault-injection capabilities. FIM is also extended with additional operators and property-based mutation testing to assess a test suite's ability to exercise the software with respect to a given property [5].

MUT4SLX is a state-of-the-art mutation testing tool capable of both automated mutant generation and execution [9, 22]. It includes a comprehensive set of parameter-based mutation operators—currently 28 for Simulink and 18 for Stateflow—modeled after realistic faults mined from an industrial bug database. It is also a unique tool supporting both requirement traceability and CI/CD pipeline compatibility. Currently, MUT4SLX is at technology readiness level (TRL) 5 and commercial readiness level (CRL) 4. This is why DUCO considered only MUT4SLX rather than other tools

¹https://nl.mathworks.com/help/simulink/ref_obsolete_blocks/signalbuilder.html (accessed 2026-03-30).

since MUT4SLX was tailored based on DUCO requirements and development pipeline.

Unlike functional operators, Chen et al. introduced 6 high-level timed mutation operators for Stateflow models, considering the importance of real-time behavior during simulation, and employed them to evaluate SimSched, a tool that utilizes model transformation [10].

BERTiMuS is a mutant generation approach that uses large language models (based on CodeBERT, a pre-trained model for programming languages) for Simulink models [31]. Valle et al. also presented an automated pipeline for Simulink-Stateflow mutant generation and evaluated eight state-of-the-art LLMs with different prompting strategies [28].

Unlike others, our work focuses on the challenges encountered and potential solutions in applying mutation testing to Simulink Cyber-Physical System models using MUT4SLX in practice, based on a pilot project with DUCO company.

3 Driving the Pilot Project

3.1 Method

The pilot project was conducted within an existing research and development project called efficient testing of control software (EFFECTS) where there was a dedicated work package on mutation testing of Simulink and Stateflow models in collaboration with DUCO company. The project lasted between 2021 and 2024. One of the deliverables of the relevant work package was a proof-of-concept tool (called MUT4SLX) and we first released our minimum-viable product (MVP) in 2023 (Q1). Since MUT4SLX was successful on providing measurable benefits (as we share in the next section) on DUCO side, we decided to further invest on mutation testing using MUT4SLX and also collected extra feature requests and comments on our proof-of-concept tool after finalizing our project. Then, we regularly share new versions addressing those features and then collect feedback from DUCO side.

3.2 Quantitative Details

The pilot was carried out on three production models taken from DUCO’s model repository, referred to here as **M1**, **M2** and **M3**; their names and internal structure cannot be disclosed (see Section 7). DUCO applied MUT4SLX internally to a considerably larger set of models and reported back a 7.5% improvement of their test suites over the first three consecutive trials of the tool. The challenges and solutions discussed in this paper emerged from that wider internal use and from continuous discussion with the DUCO engineers. These three models are the ones shared with us for joint analysis and hence the only ones we can report measurements on. M1 and M2 are Simulink® industrial control components, a saturating summation stage and a scaling PID controller. M3 is a Stateflow® controller from the heating domain. Table 2 summarises their size and the structural coverage their existing test suites achieve. The models are exercised exactly as DUCO exercises them in daily practice. A test case is a Signal Builder group that drives one input scenario, and the expected behaviour is checked by Assertion blocks embedded in the model. MUT4SLX therefore counts a mutant as killed whenever running the test suite triggers an assertion. We ran the complete operator set without any filtering, so the numbers below are raw

results. All measurements were obtained with MATLAB® R2025b on a laptop with an Intel Core i7-12700H processor (14 cores, 20 threads), 16 GB of RAM and Windows 11.

Over the three models MUT4SLX generated 459 mutants, of which 250 were killed and 209 survived, yielding mutation scores of 82.0% (M1), 41.0% (M2) and 95.7% (M3). M1 reaches 100% decision and condition coverage, yet 16 of its 89 mutants survive. The 209 surviving mutants are also the pool from which equivalent mutants have to be identified by hand, which is the challenge discussed in Section 4.1. Execution cost is a further practical concern: M2 required 3.6 hours to simulate its 324 mutants against its 21 test cases.

Table 3 contrasts each test case in isolation with the complete suite. For M1 the individual test cases kill between 44.9% and 68.5% of the mutants while the suite as a whole reaches 82.0%, so the test cases complement each other. For M2 the picture is different. Its strongest single test case already reaches 40.1% and the full suite of 21 only reaches 41.0%, with 16 of the 21 test cases scoring between 37.7% and 38.9%. The suite is therefore largely redundant with respect to fault detection, an insight that structural coverage alone did not expose. For M3 (the smallest of the three subjects) there is only 1 test case killing 44 mutants and yielding 95.7% mutation score. Also note that even there full decision, condition and MC/DC coverage leaves two of its 46 mutants alive.

3.3 Research Questions

In this paper, we report on a pilot project for a proof-of-concept model-based mutation testing tool named MUT4SLX within the context of DUCO. DUCO has a long history of model-based engineering, including a heavy emphasis on unit tests for its Simulink and Stateflow models. The pilot study was driven by two primary research questions listed below.

RQ1: Challenges *What are the challenges of applying mutation to Simulink and Stateflow models in practice?*

Motivation. We experienced several challenges when introducing a model-based mutation testing tool within the context of DUCO. While specific to the context of DUCO, we believe they are sufficiently generic to warrant attention of the research community.

Approach. We share the challenges we encountered, considering their importance and severity. We give priority to the most severe challenges (major), including some relevant minor ones in between.

RQ2: Possible Solutions *What are the possible solutions to the encountered challenges?*

Motivation. During the pilot project we addressed these challenges to work within the context of the DUCO engineering team. Here as well our lessons learned may benefit the larger research community.

Approach. Based on the feedback of the DUCO engineering team, we reviewed the existing literature relevant to the feedback, highlighted topics and considered possible solutions. Then, we explored whether it is feasible to incorporate these solutions in the context of Simulink and Stateflow, expanding MUT4SLX with extra features. We document our lessons learned.

Table 2: Characteristics of the three industrial models used in the pilot, and the structural coverage reached by their existing test suites when all test cases are active.

Model	Model type	Component under test			Test suite		Coverage (%)		
		Blocks	Subsystems	States / Trans.	Test cases	Input signals	Decision	Condition	MC/DC
M1	Simulink	49	2	N/A	5	4	100.0	100.0	N/A
M2	Simulink	255	11	N/A	21	18	69.6	60.0	N/A
M3	Stateflow	28	3	6 / 14	1	8	100.0	100.0	100.0

Abbreviations: Trans. = transitions; MC/DC = modified condition/decision coverage; N/A = the model contains no element for which the metric is defined. Coverage is measured on the component under test, not on the surrounding test model.

Table 3: Mutation score of each test case in isolation compared to the score of the complete test suite.

Test case	Killed	Score (%)	Run time	
			(s)	(min)
M1 (5 test cases)				
TC1	40	44.9	16.9	0.3
TC2	40	44.9	16.9	0.3
TC3	40	44.9	17.1	0.3
TC4	41	46.1	16.9	0.3
TC5	61	68.5	17.0	0.3
All 5 test cases	73	82.0	76.7	1.3
M2 (21 test cases)				
Weakest test case	81	25.0	242.9	4.0
Median	124	38.3	224.8	3.7
Strongest test case	130	40.1	368.2	6.1
16 of the 21 test cases score between 37.7 and 38.9				
All 21 test cases	133	41.0	12823.4	213.7
M3 (1 test case)				
TC1 (= all test cases)	44	95.7	24.8	0.4

Note: Test cases are anonymised and ordered by increasing mutation score. The mutant pool is identical for every row of a model (M1: 89, M2: 324, M3: 46 mutants). For M2 the three summary rows report the minimum, median and maximum of each column over its 21 single-test-case runs.

4 Results and Discussion

4.1 RQ1: Challenges

Equivalent Mutants (MAJOR): During our work on Simulink and Stateflow models, MUT4SLX produced equivalent mutants. Those are detected by manual analysis because MUT4SLX currently does not support automatic detection. However, we learned that the majority of the equivalent mutants stem from specific operators: Constant Replacement (CR), Relational Operator Replacement (ROR), and, as a special operator, changing the Multiplication parameter of the Product and Gain blocks. The latter is representative of the equivalent mutants challenge; hence, we explain it in more detail.

In Simulink, the gain block² accepts a single input signal and multiplies it by a constant K. The gain can be a real or complex-valued scalar, vector, or matrix. A typical mutation operator would change multiplication modes from one to another, for example, from element-wise to matrix. However, as long as K is a scalar, mutating multiplication modes makes no difference and provides the same

results, regardless of the dimension of the input signal. This causes equivalent mutants. In the Product block³, however, since multiple signals are multiplied together instead of a constant, equivalent mutants can occur in more scenarios. Changing the multiplication modes makes no difference whether all inputs are scalar or only one is non-scalar.

The fact that many of the models within DUCO contained these blocks made it very difficult to manually perform semantic analysis to confirm that these mutations are equivalent or not. For these two blocks specifically, we implemented a solution that identifies the type (such as real or complex-valued scalar, vector, or matrix) of the K constant in the Gain block and the combinations (such as two scalars, a scalar and a vector) of the input signals in the Product block, and applies filtering accordingly. Although this method is not effective for equivalent mutations that change at the block level for gain and product blocks but do not propagate to higher sub-system levels, we are analysing our solution’s applicability to equivalent mutations arising in other blocks different from gain and product blocks, since it is successful for mutations that are equivalent directly at the block level.

Requirement Traceability (MAJOR): A major challenge we identified during our pilot project is linking mutation results back to requirements. Organisations maintain their requirements in diverse formats, such as Requirements Toolbox files (.slreqx), IBM DOORS, Microsoft Word, Microsoft Excel, PDF documents, and other types supported by Simulink. Each test case is typically linked to one or more requirements, yet mutation testing tools currently do not exploit this traceability information.

Mutation Testing for Stateflow (MINOR): Mutation operators for Stateflow transitions and states are primarily implemented using a regular expression-based technique due to limitations in the Stateflow API⁴, unlike in the Simulink API. This necessity arises because DUCO’s MATLAB version requires us to use this solution. The Stateflow API in this version does not allow us to modify parameters of transitions and states in Stateflow models. This is a particular solution for a specific MATLAB version implemented in MUT4SLX. Applying these techniques to Stateflow models introduces several challenges. Different mutation types require specialized regular expressions for numeric, Boolean, logical, and mathematical operations. Ensuring the syntactically correct placement of mutated

³<https://nl.mathworks.com/help/simulink/slref/product.html> (accessed 2026-03-30).

⁴<https://nl.mathworks.com/help/stateflow/api/overview-of-the-stateflow-api.html> (accessed 2026-03-30).

²<https://nl.mathworks.com/help/simulink/slref/gain.html> (accessed 2026-03-30).

expressions is also difficult, particularly when mutations replace operators with alternatives of different string lengths and characters, for example, while replacing “<” with “>=” when the original string is “<=” results in “>==” or remove the logical negation operator (!) for enums (e.g., not in !=) while preserving the structure of the original expression. In addition, the expression context must be considered when applying certain mutations. For example, when mutating the equality operator (==), it is necessary to determine whether the expression belongs to a Boolean or numeric context in order to apply an appropriate mutation (e.g., != or >=). Therefore, mutant generation requires not only detecting the operator but also analysing the string as a whole, such as the order of symbols and characters. Furthermore, the mutation process must distinguish between expressions located in Stateflow model elements and those appearing in comments. Analysing the string as a whole and detecting comments based on a specific identifier (/*...*/) could help resolve those problems.

Testing MUT4SLX (MINOR): During the development of MUT4SLX, one of the concerns is to design smoke tests to ensure that the MUT4SLX remains functional on a model that includes all types of blocks that existing mutation operators are capable of manipulating. These tests verify that the functional integrity is preserved in MUT4SLX after integration with new features, such as new mutation operators. They are also used to verify the functionality with different MATLAB versions.

4.2 RQ2: Possible Solutions

This section provides details on the solutions we adopted and the lessons we learned while adopting them.

Tackling Equivalent Mutants: We considered several options to address the issue of equivalent mutants. Based on the literature we first considered Trivial Compiler Equivalence, the dominant technique for code level mutation testing [23]. As a method for detecting equivalent mutants, the original and mutated code are compiled at the same optimization level to produce two compiled binaries, which are then compared. If they are identical byte-by-byte, the mutant is marked as equivalent and eliminated.

In the context of Simulink and Stateflow the closest parallel to the binaries are the (.slx) files. These are binary files, but are in fact compressed XML packages; therefore, comparing the original and mutated model files would not be meaningful they will be the same because there is not compiler optimization performed. An alternative is to generate C code for both the original and modified models using model-level optimizations with Embedded Coder, then using a compiler such as gcc to perform a second round of optimization on this generated code to produce binary files, and finally compare the results. However, in the practical implementation of this process, several constraints must be taken into account, including the fact that generating code from models is likely to be significantly more expensive than compiling C code.

As an alternative, we took inspiration from using formal verification to show why a mutant is equivalent, or come up with a counterexample as a test case if not [11]. As the formal verification we chose the Simulink Design Verifier. We were also inspired by the back-to-back equivalence testing approach, which verifies that Simulink models and the code generated from them are consistent.

This type of test verifies whether the Simulink model and the code generated from it produce the same results as a large input space generated by the Simulink Design Verifier. The strategy we have implemented works as follows. A broad input space is generated on the original model using Simulink Design Verifier. Subsequently, for each surviving mutant, the original and mutated models are simulated using this input space, and a comparison is made to determine whether they produce the same outputs. If both models produce the same outputs across the entire input space, that mutant is marked as “likely equivalent” and eliminated. As a result of this analysis, test cases can be created from the inputs that kill the mutants, thereby amplifying the test suite.

One drawback we experienced is that this approach generates a very large input space for large models. This results in a large number of simulations and significant time costs, thereby reducing its practical applicability. To make this approach scale, solutions such as filtering the pool of surviving mutants rather than testing all of them could be considered.

Addressing Requirement Traceability: Based on feedback from the DUCO engineers, we implemented a prototype feature for Signal Builder-based models that leverages Simulink Requirements Toolbox (.slreqx) links. When a test case (i.e., a Signal Builder group) kills a mutant, the tool traces which requirement is linked to that group, thereby associating killed mutants with the requirements they relate to. This enables identification of which requirements are well-covered by mutation testing and which may need additional test effort.

In our current tool prototype this traceability support is limited to a Signal Builder and the Simulink Requirements Toolbox. There are other test execution tools within the Simulink and Stateflow ecosystem and there are many requirement document types used across organizations. Therefore, for real-world adoption, a customized integration tailored to an organization’s specific toolchain is needed.

Handling Mutation Testing for Stateflow: To ensure syntactic integrity during operator replacement, our approach leverages regular expressions (regex) that explicitly target the whitespace delimiters surrounding operators. By capturing an operator along with its preceding and succeeding whitespaces (e.g., recognizing “<” as “< ”), the tool safely substitutes it with alternatives of varying lengths, such as “<=” or “>”, completely avoiding malformed constructs. Beyond syntactic correctness, contextual ambiguities—such as whether an equality operator (==) operates in a Boolean or numeric context—are resolved by analyzing the adjacent lexical scope for explicit Boolean literals (e.g., true or false). The expression is treated as logical if such literals are present, and numeric otherwise. Additionally, to avoid mutating code within comments, a pre-processing step delineates comment intervals across the code sequence. By explicitly excluding these blocks from the regex search space, the generation of equivalent or ineffectual mutants is effectively mitigated.

Effective Testing MUT4SLX: To achieve this, a diverse set of smoke tests is developed to cover common and representative Simulink and Stateflow use cases while maintaining coverage of existing functionality. To do this, we created several artificial models and extended them as new features were developed, which needed

to be tested. This also requires continuously extending test capabilities for newly introduced mutation operators and updating tests when existing operators are refactored. To support this process, a Continuous Integration and Continuous Deployment (CI/CD) pipeline is employed using Jenkins, which automates the execution of tests for both existing and newly implemented features.

5 Lessons Learned and Practical Insights

This section presents our lessons learned from our pilot project, drawing on the challenges already shared and possible solutions.

Lesson 1: *Equivalent mutants concentrate in a small set of operators, so operator-specific filters outperform general-purpose detection.*

When we manually analysed the equivalent mutants generated by MUT4SLX, we discovered that the majority of them originated from only three operators: Constant Replacement (CR), Relational Operator Replacement (ROR), and mutation of the Multiplication parameter in the Gain/Product blocks. This led us to develop a dimensionality-tracing filter that works at the block level. This inexpensive, operator-specific solution eliminated many equivalent mutants without ever introducing them into the simulation, whereas general approaches based on Trivial Compiler Equivalence or Design Verifier induced unacceptable execution times and licensing dependencies for the same mutant pool.

Lesson 2: *Code-level equivalent mutant detection techniques do not transfer to model-level without substantial adaptation.*

We attempt to use the Trivial Compiler Equivalence (TCE) method on Simulink and Stateflow models. While (.slx) files are comparable at the byte level, they are actually compressed XML packets, so the same semantics means the same binary assumption on which Trivial Compiler Equivalence is based is not directly applicable. Alternatively, when we tried generating C code with Embedded Coder and comparing the binary after gcc optimisation, we found that code generation was much more expensive than C compilation, that Embedded Coder introduced a license dependency, and that generation was not fully deterministic. We learned that a technique that is straightforward at the code level is not easily applicable at the model level.

Lesson 3: *Requirement traceability in practice is an integration problem; better not consider it a tool feature.*

In industry, requirements are stored in diverse formats, such as (.slreqx), IBM DOORS, Microsoft Word, Excel, and PDF. When we developed a prototype traceability feature for the Signal Builder and (.slreqx) combination, even for this single combination, a point to point integration was costly to implement. More importantly, the solution could not be directly transferred to other test execution tools or other requirement formats. We learned that designing traceability as a single feature integrated into MUT4SLX was impractical. The more realistic approach is an integration layer that can be customised to the organisation’s existing toolchain. This is

why we positioned our (.slreqx) support as a “proof-of-concept” rather than a “general solution”.

Lesson 4: *Fine-tune the mutation operators based on the bug database of the software team.*

Mutation operators for Simulink and Stateflow can be defined and implemented in different layers based on their classes, as shown in Table 1. The selection of classes and mutation operators needs to be determined based on company-specific requirements. Then, when an appropriate class is selected, the mutation operators must be customised using the software team’s bug database. This makes them more targeted, relevant and impactful. The mutation operators for MUT4SLX are designed and developed based on this approach, considering the needs and requirements of DUCO and fine-tuned using their bug database.

6 Conclusion

Based on shift-left testing principles, several mutation testing tools for Simulink and Stateflow models have been presented. While these proof-of-concept tools demonstrate the technical feasibility of model-based mutation testing, actual experience reports on how the corresponding tools would work in practice are still lacking. In this paper, we analyse the challenges we encountered and the lessons we learned when incorporating mutation testing in the context of DUCO. The main challenges, ranked by severity, are equivalent mutants, requirement traceability, and mutation testing for Stateflow.

7 Mandatory Data Availability Statement

The authors confirm that the data supporting the findings and experiments of this study are subject to third-party restrictions and cannot be shared due to the high sensitivity of our industrial partner’s data. The reader could reach out to the authors for further supporting resources.

8 Acknowledgments

This work is supported by (a) the Flanders Innovation & Entrepreneurship (VLAIO) under grant number HBC.2021.0010 entitled “EFFECTS”; (b) the Research Foundation Flanders (FWO) under grant number S000323N entitled “Basecamp Zero”; (d) Flanders Make, the strategic research centre for the manufacturing industry. (e) the Agency for Innovation & Entrepreneurship (VLAIO) via the project “TTRUST” under grant number HBC20230612. (f) the Industrial Research Fund (IOF) via the project “MUT4SLX” under grant number 54104.

References

- [1] Bernhard K Aichernig, Harald Brandl, Elisabeth Jöbstl, Willibald Krenn, Rupert Schlick, and Stefan Tiran. 2015. Killing strategies for model-based mutation testing. *Software Testing, Verification and Reliability* 25, 8 (2015), 716–748. doi:10.1002/stvr.1522
- [2] Aitor Arrieta, Urtzi Markiegi, and Leire Etzeberria. 2017. Towards mutation testing of configurable Simulink models: A product line engineering perspective. *Jornadas de Ingeniera del So ware y Bases de Datos* (2017).
- [3] Richard Baker and Ibrahim Habli. 2012. An empirical evaluation of mutation testing for improving the test quality of safety-critical software. *IEEE Transactions on Software Engineering* 39, 6 (2012), 787–805. doi:10.1109/TSE.2012.56
- [4] Ezio Bartocci, Leonardo Mariani, Dejan Ničković, and Drishti Yadav. 2022. FIM: fault injection and mutation for Simulink. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1716–1720. doi:10.1145/3540250.3558932
- [5] Ezio Bartocci, Leonardo Mariani, Dejan Ničković, and Drishti Yadav. 2023. Property-based mutation testing. In *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 222–233. doi:10.1109/ICST57152.2023.00029
- [6] Fevzi Belli, Christof J Budnik, Axel Hollmann, Tugkan Tuglular, and W Eric Wong. 2016. Model-based mutation testing—approach and case studies. *Science of Computer Programming* 120 (2016), 25–48. doi:10.1016/j.scico.2016.01.003
- [7] Angelo Brillout, Nannan He, Michele Mazzucchi, Daniel Kroening, Mitra Purandare, Philipp Rümmer, and Georg Weissenbacher. 2009. Mutation-based test case generation for Simulink models. In *International Symposium on Formal Methods for Components and Objects*. Springer, 208–227. doi:10.1007/978-3-642-17071-3_11
- [8] Timothy A Budd, Richard A DeMillo, Richard J Lipton, and Frederick G Sayward. 1980. Theoretical and empirical studies on using program mutation to test the functional correctness of programs. In *Proceedings of the 7th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 220–233. doi:10.1145/567446.567468
- [9] Halil Ibrahim Ceylan, Onur Kilincceker, Mutlu Beyazit, and Serge Demeyer. 2023. MUT4SLX: Fast mutant generation for Simulink. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2086–2089. doi:10.1109/ASE56229.2023.00093
- [10] Jian Chen, Manar H Alalfi, and Thomas R Dean. 2024. Timed Model-Based Mutation Operators for Simulink Models. In *2024 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 263–272. doi:10.1109/ICSTW60967.2024.00055
- [11] Serge Demeyer and Reiner Hähnle. 2025. Equivalent Mutants: Deductive Verification to the Rescue. In *2025 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 326–336. doi:10.1109/ICSTW64639.2025.10962501
- [12] Richard A DeMillo, Richard J Lipton, and Frederick G Sayward. 2006. Hints on test data selection: Help for the practicing programmer. *Computer* 11, 4 (2006), 34–41. doi:10.1109/C-M.1978.218136
- [13] Xavier Devroey, Gilles Perrouin, Mike Papadakis, Axel Legay, Pierre-Yves Schobbens, and Patrick Heymans. 2016. Featured model-based mutation analysis. In *Proceedings of the 38th International Conference on Software Engineering*. 655–666. doi:10.1145/2884781.2884821
- [14] Carlos A González, Mojtaba Varmazyar, Shiva Nejati, Lionel C Briand, and Yago Isasi. 2018. Enabling model testing of cyber-physical systems. In *Proceedings of the 21th ACM/IEEE international conference on model driven engineering languages and systems*. 176–186. doi:10.1145/3239372.3239409
- [15] Le Thi My Hanh and Nguyen Thanh Binh. 2012. Mutation operators for Simulink models. In *2012 Fourth International Conference on Knowledge and Systems Engineering*. IEEE, 54–59. doi:10.1109/KSE.2012.22
- [16] Le Thi My Hanh, Nguyen Thanh Binh, and Khuat Thanh Tung. 2016. A Novel Fitness function of metaheuristic algorithms for test data generation for Simulink models based on mutation analysis. *Journal of Systems and Software* 120 (2016), 17–30. doi:10.1016/j.jss.2016.07.001
- [17] Edward A Lee. 2015. The past, present and future of cyber-physical systems: A focus on models. *Sensors* 15, 3 (2015), 4837–4869. doi:10.3390/s150304837
- [18] Reza Matinnejad, Shiva Nejati, Lionel C Briand, and Thomas Bruckmann. 2016. Automated test suite generation for time-continuous Simulink models. In *Proceedings of the 38th International Conference on Software Engineering*. 595–606. doi:10.1145/2884781.2884797
- [19] Reza Matinnejad, Shiva Nejati, Lionel C Briand, and Thomas Bruckmann. 2018. Test generation and test prioritization for simulink models with dynamic behavior. *IEEE Transactions on Software Engineering* 45, 9 (2018), 919–944. doi:10.1109/TSE.2018.2811489
- [20] Suzanne Miller and Don Firesmith. 2021. Four types of shift left testing. (2021).
- [21] Manju Nanda, J Jayanthi, V Madhan, and Sameer Nayak. 2017. An Integrated Mutation Analysis Tool (Imat) For Simulink Models and Their Ideal Implementation. *Journal of Aerospace Sciences and Technologies* (2017), 361–372. doi:10.61653/joast.v6i93.2017.278
- [22] Sien Nuyens, Halil Ibrahim Ceylan, Onur Kilincceker, Mutlu Beyazit, and Serge Demeyer. 2024. MUT4SLX: extensions for mutation testing of Stateflow models. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering-Companion (SANER-C)*. IEEE, 215–218. doi:10.1109/SANER-C62648.2024.00035
- [23] Mike Papadakis, Yue Jia, Mark Harman, and Yves Le Traon. 2015. Trivial compiler equivalence: A large scale empirical study of a simple, fast and effective equivalent mutant detection technique. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. IEEE, 936–946. doi:10.1109/ICSE.2015.103
- [24] Ingo Pill, Ivan Rubil, Franz Wotawa, and Mihai Nica. 2016. Simultate: A toolset for fault injection and mutation testing of Simulink models. In *2016 IEEE Ninth International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 168–173. doi:10.1109/ICSTW.2016.21
- [25] Strategic Planning. 2002. The economic impacts of inadequate infrastructure for software testing. *National Institute of Standards and Technology* 1, 2002 (2002), 1.
- [26] Rakesh Rana, Mirosław Staron, Christian Berger, Jörgen Hansson, Martin Nilsson, and Fredrik Törner. 2013. Early verification and validation according to ISO 26262 by combining fault injection and mutation testing. In *International Conference on Software Technologies*. Springer, 164–179. doi:10.1007/978-3-662-44920-2_11
- [27] Matthew Stephan, Manar H Alalfi, and James R Cordy. 2014. Towards a taxonomy for Simulink model mutations. In *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation Workshops*. IEEE, 206–215.
- [28] Pablo Valle, Shaikat Ali, and Aitor Arrieta. 2026. Exploring the Potential of Large Language Models in Simulink-Stateflow Mutant Generation. In *Proceedings of the 2026 IEEE/ACM Third International Conference on AI Foundation Models and Software Engineering (FORGE '26)*. Association for Computing Machinery, New York, NY, USA, 99–109. doi:10.1145/3793655.3793737
- [29] Drishti Yadav, Claudio Mandrioli, Ezio Bartocci, and Domenico Bianculli. 2025. Fault Injection for Simulink-based CPS Models: Insights and Future Directions. In *ASE 2025*. IEEE. doi:10.1109/ASE63991.2025.00326
- [30] Yuan Zhan and John A Clark. 2005. Search-based mutation testing for Simulink models. In *Proceedings of the 7th annual conference on Genetic and evolutionary computation*. 1061–1068. doi:10.1145/1068009.1068188
- [31] Jingfan Zhang, Delaram Ghobari, Mehrdad Sabetzadeh, and Shiva Nejati. 2025. Simulink mutation testing using CodeBERT. In *2025 IEEE/ACM International Conference on Automation of Software Test (AST)*. IEEE, 24–28. doi:10.1109/AST66626.2025.00009